

Worked Example of the Metrics

1 Question and Retrieved Context

QUESTION

My mock never applies, and the real API response shows up instead. Code:

```
await page.goto('https://demo.playwright.dev/api-mocking');
await page.route('**/api/v1/fruits', async route => {
  await route.fulfill({ json: [{ name: 'Strawberry', id: 21 }] });
});
await expect(page.getByText('Strawberry')).toBeVisible(); // fails
What did I do wrong?
```

REFERENCE ANSWER

You registered the route after navigating, so the request already fired during `page.goto()` before the mock was active. Register the route before navigating, that is, mock the API call before `page.goto()`.

The traversal returned three passages, listed here in the order it ranked them.

PASSAGE 1 - Missing network events and Service Workers

"If you're using Playwright's native `browserContext.route()` and `page.route()`, and it appears network events are missing, disable Service Workers by setting `serviceWorkers` to 'block'. It might be that you are using a mock tool such as Mock Service Worker (MSW) [...] making them invisible to [...] `page.route()`."

PASSAGE 2 - Mock API requests

```
"The following code will intercept all the calls to **/api/v1/fruits and will return a custom response instead. [...]\n// Mock the api call before navigating\nawait page.route('**/api/v1/fruits', async route => { ... });\n// Go to the page\nawait page.goto('https://demo.playwright.dev/api-mocking');"
```

PASSAGE 3 - URL matching with glob patterns

"Playwright uses simplified glob patterns for URL matching in network interception methods [...] A single `*` matches any characters except `/`, a double `**` matches any characters including `/`."

GENERATED ANSWER

You are calling `page.goto` before setting up the route. To ensure the mock is applied, you must call `page.route` to intercept the API call before navigating to the page. Additionally, if you are using a tool like Mock Service Worker (MSW), it may be taking over network requests and making them invisible to `page.route`. If this is the case, you should disable Service Workers by setting `serviceWorkers` to 'block' in your context configuration.

5.3.2 Retrieval Metrics

The judge inspects each retrieved passage in rank order and marks whether it was useful in arriving at the reference answer. Only the second passage carries the ordering fact the reference answer turns on, so the verdict vector of Equation (8) is $v = (0, 1, 0)$ over $n = 3$ passages. Passage 1 describes a different cause of a mock not applying and passage 3 concerns pattern syntax that the query's pattern already satisfies, so neither contributes.

Precision@1 = 0/1 = 0, Precision@2 = (0 + 1)/2 = 0.5, Precision@3 = (0 + 1 + 0)/3 = 1/3.

The denominator of Equation (8) is the number of retrieved passages the judge marked useful, here $\sum_k v_k = 1$, not the number of passages in the corpus that could have answered the question. Passages scoring zero drop out of the numerator, so only the second term survives:

$$\text{Context Precision} = [(0)(0) + (0.5)(1) + (1/3)(0)] / 1 = 0.5.$$

Placing the one useful passage second rather than first is what costs half the score. Had the same three passages been returned in the opposite order, the value would have been 1.0 with no change to what was retrieved.

For Context Recall the judge decomposes the reference answer into $J = 2$ statements, one diagnosing the cause and one prescribing the fix, and checks each against the retrieved passages. The prescriptive statement is attributable to passage 2 and the diagnostic one is not stated anywhere in the retrieved text, giving $u = (0, 1)$ and

$$\text{Context Recall} = (1/2)(0 + 1) = 0.5.$$

5.3.3 Generation Metrics

Answer Accuracy asks the judge to rate the generated answer against the reference twice, with the two answers swapped on the second pass. The first pass returned $g_1 = 4$, exact agreement, since the generated answer states the correct cause and the correct fix. The second returned $g_2 = 2$, partial agreement, because in that direction the material about Service Workers has no counterpart in the reference. By Equation (10),

$$\text{Answer Accuracy} = (4 + 2) / 8 = 0.75.$$

Factual Correctness compares the two answers claim by claim. The generated answer decomposes into four claims, of which the reference entails the first two and neither entails nor contradicts the last two, so $TP = 2$ and $FP = 2$. Both unentailed claims came from passage 1, which is how a retrieval error propagates into this score. The reference decomposes into four claims here, all of them covered by the generated answer, so $FN = 0$. Applying Equation (11),

$$P_c = 2/4 = 0.5, R_c = 2/2 = 1.0, F_1 = 2(0.5)(1.0)/(0.5 + 1.0) \approx 0.667.$$

The reference answer yields two units under Context Recall and four under Factual Correctness because the framework applies a different decomposition to each. Context Recall splits the reference into statements and tests attribution to the retrieved context, while Factual Correctness splits it into atomic claims for natural language inference against the generated answer. The counts are not expected to agree, and neither metric reads the other's segmentation.

Two metrics have no arithmetic to display. Helpfulness is the rubric level the judge selects, so the label is already the score. ROUGE-L is computed from token overlap without a judge, so no labels enter it.